

Interior Point Method for Sparse Recovery

A MATLAB Implementation of ℓ_1 -Minimization via Linear Programming

Carlota Arzúa Alonso

Mathematics and Computer Science
DePaul University

Technical Project Report

July 2026

Abstract

This report presents a MATLAB implementation of an Interior Point Method for recovering a sparse vector from an underdetermined linear system. The problem begins with a measurement matrix $A_0 \in \mathbb{R}^{52 \times 128}$ and a measurement vector $b = A_0 x$, where the unknown vector $x \in \mathbb{R}^{128}$ contains only five nonzero entries. Recovery is formulated as the convex optimization problem $\min_x \|x\|_1$ subject to $A_0 x = b$. By decomposing the signed vector into nonnegative positive and negative components, the problem is converted into a linear program with 256 decision variables. A custom Newton-style Interior Point iteration is then implemented directly in MATLAB without calling the built-in `linprog` solver. In the reported run, the method recovered all five nonzero locations correctly, achieved an ℓ_2 recovery error of approximately 0.004598, a relative ℓ_2 error of 0.055880%, and a maximum absolute error of 0.002500. The project demonstrates the connection between convex optimization, sparse recovery, numerical linear algebra, and scientific software design.

Contents

1	Introduction	1
2	Problem Formulation	1
2.1	Underdetermined measurement model	1
2.2	ℓ_1 -minimization objective	2
2.3	Conversion to a linear program	2
3	Interior Point Method	3
3.1	Algorithm parameters	3
3.2	Adjusted diagonal matrix	3
3.3	Multiplier system	3
3.4	Newton direction and update	3
4	MATLAB Implementation	4
4.1	Software organization	4
4.2	Deterministic measurement matrix	4
4.3	Initial feasible estimate	4
4.4	Conversion to nonnegative variables	5
4.5	Core solver loop	5
5	Experimental Setup	6
5.1	Dimensions and sparsity	6
5.2	Thresholding and signed reconstruction	6
5.3	Evaluation metrics	6
6	Results	7
6.1	Overall recovery quality	7
6.2	Recovered nonzero values	7
6.3	Visual comparison	7

7	Numerical and Software Engineering Considerations	8
7.1	Linear solves instead of explicit inverses	8
7.2	Regularization of the diagonal matrix	9
7.3	Separation of support and value accuracy	9
7.4	Deterministic reproducibility	9
7.5	Direct algorithm implementation	9
8	Limitations and Future Work	9
8.1	Fixed problem instance	9
8.2	Noisy measurements	10
8.3	Fixed algorithm parameters	10
8.4	Pure Newton step	10
8.5	Benchmarking	10
9	Conclusion	10
10	Project Background and Provenance	10
A	Algorithm Summary	11
B	Selected MATLAB Source	11

1 Introduction

Sparse recovery asks whether a high-dimensional vector can be reconstructed from a smaller number of linear measurements when most of its entries are zero. This setting appears naturally in compressed sensing, inverse problems, signal reconstruction, and other applications where the underlying object is structured but the available observations are limited.

The project considered here uses a matrix

$$A_0 \in \mathbb{R}^{52 \times 128}$$

and an unknown vector

$$x \in \mathbb{R}^{128}.$$

Only the measurement vector

$$b = A_0 x$$

is observed. Because there are fewer equations than unknowns, the system is underdetermined: many vectors may satisfy the equality constraint. The additional assumption is that the true vector is *5-sparse*, meaning exactly five of its 128 entries are nonzero.

The central idea is to recover the sparse solution by minimizing the ℓ_1 norm subject to the measurement constraint. The resulting convex problem can be reformulated as a linear program and solved with an Interior Point Method. Interior point algorithms form a major class of methods for linear programming and are studied as alternatives to simplex-based approaches; the course project was based on the treatment by Robere [1].

The goals of this project are:

1. formulate sparse recovery as an ℓ_1 -minimization problem;
2. convert the signed-variable problem into a standard nonnegative linear program;
3. implement the specified Interior Point iteration directly in MATLAB;
4. evaluate both support recovery and numerical accuracy; and
5. organize the implementation using reproducible numerical-computing practices.

2 Problem Formulation

2.1 Underdetermined measurement model

Let

$$A_0 \in \mathbb{R}^{52 \times 128}, \quad x \in \mathbb{R}^{128}, \quad b \in \mathbb{R}^{52},$$

with

$$A_0 x = b.$$

Since $52 < 128$, the equality constraint alone does not determine a unique vector x . The recovery objective therefore uses sparsity as additional structure.

The true vector contains five nonzero values at indices 30, 43, 55, 107, and 109. The values are summarized in Table 1.

Table 1: Nonzero entries of the true sparse vector.

Index	True value
30	-4.3000
43	3.2000
55	3.2000
107	-4.3000
109	3.2000

2.2 ℓ_1 -minimization objective

The sparse recovery problem is posed as

$$\min_x \|x\|_1 \quad \text{subject to} \quad A_0 x = b. \quad (1)$$

The ℓ_1 norm is

$$\|x\|_1 = \sum_{i=1}^{128} |x_i|.$$

Unlike the equality constraint, this objective prefers solutions with smaller total absolute magnitude and is commonly used as a convex surrogate for sparsity.

2.3 Conversion to a linear program

The vector x can contain both positive and negative entries. Introduce nonnegative vectors x^+ and x^- such that

$$x = x^+ - x^-, \quad x^+, x^- \geq 0. \quad (2)$$

Define the 256-dimensional nonnegative variable

$$z = \begin{bmatrix} x^+ \\ x^- \end{bmatrix} \in \mathbb{R}^{256}$$

and the enlarged matrix

$$A = \begin{bmatrix} A_0 & -A_0 \end{bmatrix} \in \mathbb{R}^{52 \times 256}.$$

Then

$$Az = A_0 x^+ - A_0 x^- = A_0 x = b.$$

With $c = \mathbf{1} \in \mathbb{R}^{256}$, the objective becomes

$$c^T z = \mathbf{1}^T x^+ + \mathbf{1}^T x^-,$$

which represents the ℓ_1 objective under the positive-negative decomposition. The linear program is therefore

$$\min_z c^T z \quad \text{subject to} \quad Az = b, \quad z \geq 0. \quad (3)$$

3 Interior Point Method

3.1 Algorithm parameters

The implementation follows the specified Interior Point iteration with the parameters in Table 2.

Table 2: Interior Point Method parameters.

Parameter	Meaning	Value
μ_0	Initial barrier parameter	10
ρ	Barrier reduction factor	0.6
ϵ	Diagonal adjustment	0.001
α	Newton step size	1
—	Maximum outer iterations	20
—	Maximum inner iterations	100
—	Inner stopping tolerance	10^{-9}

At the beginning of each outer iteration after the first,

$$\mu \leftarrow \rho\mu. \quad (4)$$

This gradually changes the barrier parameter over the sequence of outer iterations.

3.2 Adjusted diagonal matrix

For the current estimate z , define

$$X = \text{diag}(z) + \epsilon I. \quad (5)$$

The small diagonal adjustment ϵI is included in the specified method because the matrix system involving AX^2A^T can be singular or numerically problematic. In this implementation, $\epsilon = 0.001$.

3.3 Multiplier system

The multiplier vector λ is found by solving

$$(AX^2A^T)\lambda = AX^2c - \mu AXe, \quad (6)$$

where e is the vector of ones.

The MATLAB implementation uses the backslash operator:

```
1 lambda = lhs \ rhs;
```

Listing 1: Linear solve for the multiplier vector.

This solves the linear system directly without explicitly forming a matrix inverse.

3.4 Newton direction and update

The Newton direction is

$$p = Xe + \frac{1}{\mu}X^2(A^T\lambda - c). \quad (7)$$

The estimate is updated as

$$z \leftarrow z + \alpha p. \quad (8)$$

The project uses $\alpha = 1$, corresponding to a pure Newton step.

The inner iteration stops when

$$\|p\|_2 < 10^{-9}. \quad (9)$$

Otherwise, the method continues until the maximum of 100 inner iterations is reached.

4 MATLAB Implementation

4.1 Software organization

The portfolio version separates the experiment into four main components:

- `run_demo.m`: coordinates the experiment and exports results;
- `build_sparse_recovery_problem.m`: constructs the deterministic matrix, sparse vector, measurements, and initial point;
- `interior_point_solver.m`: implements the custom optimization iteration; and
- `recovery_metrics.m`: computes numerical and support-recovery metrics.

This decomposition separates problem construction, optimization, evaluation, and experiment execution. It also makes the solver easier to inspect independently from the data setup.

4.2 Deterministic measurement matrix

The measurement matrix is derived from a recursively constructed 128×128 structured sign matrix. A fixed set of 52 rows is selected to form A_0 . Because the construction is deterministic, repeated runs use the same recovery problem.

The true sparse vector is initialized by

```

1 x_true = zeros(128, 1);
2 locations = [109, 107, 55, 30, 43];
3 values = [3.2, -4.3, 3.2, -4.3, 3.2];
4 x_true(locations) = values;
5
6 b = A0 * x_true;
```

Listing 2: Ground-truth sparse vector construction.

4.3 Initial feasible estimate

A signed initial estimate is computed using

$$x_{\text{initial}} = A_0^T (A_0 A_0^T)^{-1} b.$$

In the refactored implementation, the inverse is not formed explicitly. Instead, MATLAB solves

$$(A_0 A_0^T) y = b$$

and then sets

$$x_{\text{initial}} = A_0^T y.$$

The corresponding code is

```

1 gram_matrix = A0 * A0';
2 y = gram_matrix \ b;
3 x_initial = A0' * y;

```

Listing 3: Initialization using a linear solve.

This is preferable to materializing `inv(gram_matrix)` when the actual task is to solve a linear system.

4.4 Conversion to nonnegative variables

The signed initial estimate is converted to the nonnegative variable z_0 by

```

1 x_plus = max(x_initial, 0);
2 x_minus = max(-x_initial, 0);
3 z0 = [x_plus; x_minus];

```

Listing 4: Positive-negative decomposition of the initial estimate.

The enlarged LP matrix and objective are

```

1 A = [A0, -A0];
2 c = ones(size(A, 2), 1);

```

Listing 5: Linear-program construction.

4.5 Core solver loop

The main iteration constructs X , forms the multiplier system, solves for λ , computes the Newton direction, updates the estimate, and checks convergence.

```

1 for outer = 1:options.max_outer_iterations
2     if outer > 1
3         mu = mu * options.rho;
4     end
5
6     for inner = 1:options.max_inner_iterations
7         X = diag(z) + options.epsilon * I;
8         X2 = X * X;
9
10        lhs = A * X2 * A';
11        rhs = A * X2 * c - mu * A * X * e;
12        lambda = lhs \ rhs;
13
14        p = X * e + (1 / mu) * X2 * (A' * lambda - c);
15        z = z + options.alpha * p;
16
17        if norm(p, 2) < options.tolerance
18            break;
19        end
20    end
21 end

```

Listing 6: Core Interior Point iteration.

Importantly, the solver does not call MATLAB's `linprog`. The numerical method itself is implemented directly.

5 Experimental Setup

5.1 Dimensions and sparsity

The experiment uses:

- signal dimension: 128;
- number of measurements: 52;
- number of nonzero entries: 5;
- LP decision variables after splitting: 256.

The measurement vector is generated exactly from the true vector:

$$b = A_0 x_{\text{true}}.$$

Thus, the experiment evaluates recovery in a noiseless measurement setting.

5.2 Thresholding and signed reconstruction

After optimization, entries of the 256-dimensional LP variable whose magnitude is below 0.01 are set to zero. The signed vector is then reconstructed as

$$x_{\text{recovered}} = z_{1:128} - z_{129:256}. \quad (10)$$

The thresholding choice is preserved from the project method and is documented explicitly because it affects support recovery and reported error metrics.

5.3 Evaluation metrics

Let

$$r = x_{\text{recovered}} - x_{\text{true}}.$$

The report uses the following metrics:

ℓ_2 recovery error

$$E_2 = \|r\|_2. \quad (11)$$

Relative ℓ_2 error

$$E_{\text{rel}} = \frac{\|r\|_2}{\|x_{\text{true}}\|_2}. \quad (12)$$

Maximum absolute error

$$E_{\infty} = \max_i |r_i|. \quad (13)$$

Support recovery The support of a vector is the set of indices corresponding to nonzero entries. Correct support recovery counts how many true nonzero indices are also present in the recovered support.

6 Results

6.1 Overall recovery quality

The reported run recovered the correct five nonzero locations. Table 3 summarizes the main results.

Table 3: Reported sparse-recovery results.

Metric	Result
Signal dimension	128
Measurements	52
True nonzero entries	5
Correct support entries	5 / 5
ℓ_2 recovery error	0.004598
Relative ℓ_2 error	0.055880%
Maximum absolute error	0.002500
Outer iterations	20
Sample runtime	0.257777 s

The runtime is taken from the original MATLAB run and is machine-dependent. The error metrics are computed from the thresholded recovered vector shown in that run.

6.2 Recovered nonzero values

Table 4 compares the true and recovered values at the five nonzero indices.

Table 4: Recovered nonzero entries.

Index	True value	Recovered value	Absolute error
30	-4.3000	-4.2975	0.0025
43	3.2000	3.1982	0.0018
55	3.2000	3.1979	0.0021
107	-4.3000	-4.2982	0.0018
109	3.2000	3.1980	0.0020

All five support locations are recovered. The largest absolute discrepancy among these entries is 0.0025.

6.3 Visual comparison

Figure 1 compares the true and recovered sparse vectors over all 128 indices. Because the estimates are close, the recovered markers nearly overlap the true nonzero entries.

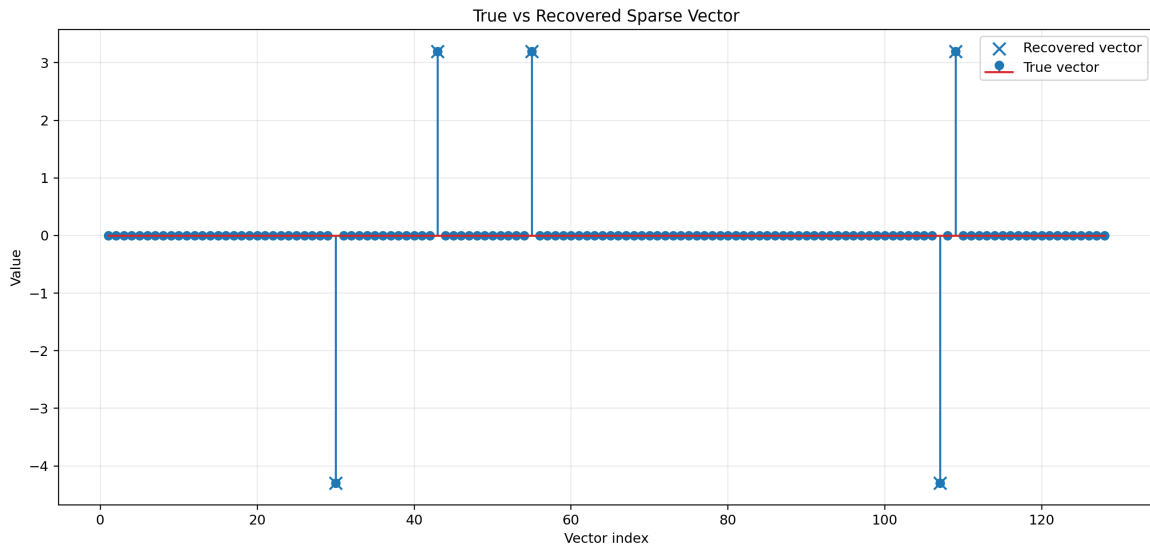


Figure 1: Comparison of the true and recovered sparse vectors.

Figure 2 shows the absolute errors at the recovered nonzero locations.

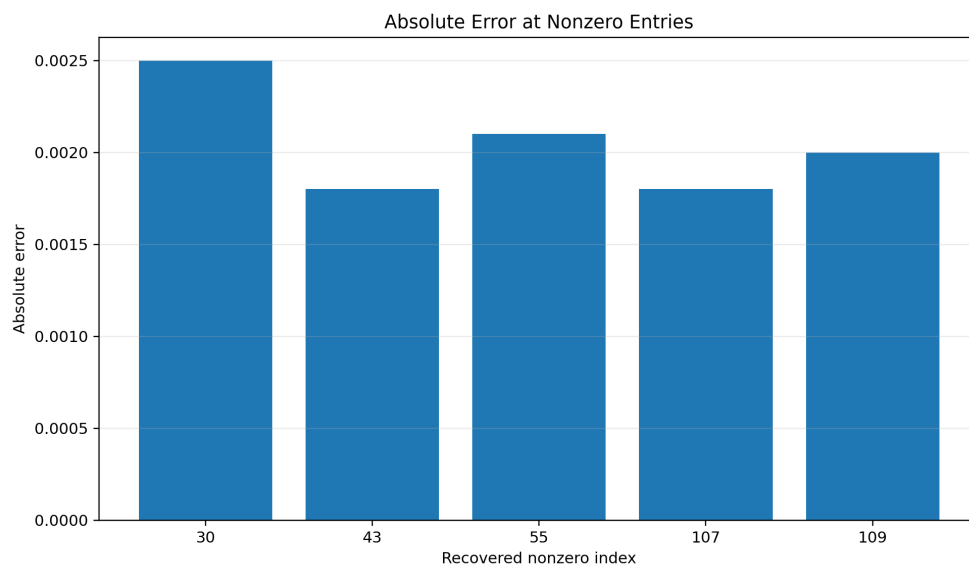


Figure 2: Absolute error at the five nonzero indices.

7 Numerical and Software Engineering Considerations

7.1 Linear solves instead of explicit inverses

A key numerical improvement in the refactored implementation is the use of MATLAB's backslash operator. The code solves systems such as

$$(A_0 A_0^T) y = b$$

rather than explicitly computing $(A_0 A_0^T)^{-1}$. This is clearer and better aligned with standard numerical linear algebra practice.

7.2 Regularization of the diagonal matrix

The update uses

$$X = \text{diag}(z) + \epsilon I.$$

The adjustment prevents the diagonal matrix from directly inheriting zero entries from the current iterate. The project specification notes that the matrix AX^2A^T is often singular, motivating the use of the ϵI adjustment.

7.3 Separation of support and value accuracy

A sparse-recovery method can identify the correct nonzero locations but still estimate their values poorly. Conversely, it can approximate several values well while missing part of the true support. Reporting both support recovery and numerical error therefore provides a more informative evaluation than either measure alone.

7.4 Deterministic reproducibility

The matrix construction, selected rows, support locations, support values, and algorithm parameters are fixed. This makes the experiment reproducible apart from machine-dependent runtime and potential environment-level floating-point differences.

7.5 Direct algorithm implementation

The custom solver does not call a black-box LP routine. This is important for the educational and technical purpose of the project: the implementation exposes the matrix system, multiplier solve, Newton direction, barrier schedule, and stopping criterion directly.

8 Limitations and Future Work

The project produces strong results on the specified deterministic problem instance, but several limitations remain.

8.1 Fixed problem instance

The reported experiment uses one deterministic 5-sparse vector and one deterministic measurement matrix. A broader evaluation would vary:

- sparse support locations;
- nonzero amplitudes;
- number of measurements;
- sparsity level; and
- measurement-matrix construction.

8.2 Noisy measurements

The current experiment uses exact measurements $b = A_0 x_{\text{true}}$. Real data may include noise. A natural extension would replace the hard equality constraint with a noise-aware formulation, such as allowing a bounded residual.

8.3 Fixed algorithm parameters

The method uses fixed values $\mu_0 = 10$, $\rho = 0.6$, and $\epsilon = 0.001$. The course specification explicitly identifies dynamic adjustment of these quantities as beyond the original scope. Future work could study how parameter schedules affect convergence speed, stability, and recovery accuracy.

8.4 Pure Newton step

The implementation uses $\alpha = 1$. A more general solver could use line search or fraction-to-the-boundary rules to control step length and improve robustness across broader problem classes.

8.5 Benchmarking

Another extension would compare the custom solver with MATLAB's `linprog` or other optimization methods. Such a comparison should distinguish between educational algorithm implementation and production-grade optimization performance.

9 Conclusion

This project demonstrates how sparse recovery can be formulated as a linear program and solved using a custom Interior Point Method. Starting from an underdetermined system with 52 measurements and 128 unknowns, the signed vector is decomposed into nonnegative components, producing a 256-variable linear program. The MATLAB implementation directly carries out the barrier update, adjusted diagonal construction, multiplier-system solve, Newton direction computation, and convergence check.

The reported result recovered all five nonzero locations of the true sparse vector and achieved a relative ℓ_2 error of approximately 0.055880%. These results show that the implemented method successfully solved the specified recovery problem with high numerical accuracy.

Beyond the numerical result, the project illustrates several broader skills: translating mathematical pseudocode into working software, reasoning about matrix dimensions and linear systems, avoiding unnecessary explicit inverses, measuring numerical error, and organizing a scientific-computing codebase into reproducible components. The project therefore sits at the intersection of applied mathematics, numerical optimization, and software engineering.

10 Project Background and Provenance

This report is based on a MAT 387/487 university numerical optimization project. The course materials provided the project problem, deterministic setup conventions, algorithm specification, parameter values, and MATLAB scaffold surrounding the student implementation section. The Interior Point iteration was implemented for the project and later reorganized into modular MATLAB components with additional metrics, visualizations, and technical documentation.

The course project itself was based on Robert Robere’s 2012 report on Interior Point Methods and Linear Programming [1]. The purpose of this provenance statement is to distinguish the supplied problem context and scaffold from the implementation, refactoring, evaluation, and documentation represented in the portfolio version.

References

- [1] Robert Robere. *Interior Point Methods and Linear Programming*. University of Toronto, December 13, 2012.
- [2] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

A Algorithm Summary

Given c , A , b , and an initial estimate z_0 :

1. Initialize $\mu = 10$, $\rho = 0.6$, $\epsilon = 0.001$, and $z = z_0$.
2. For 20 outer iterations:
 - (a) If this is not the first outer iteration, set $\mu \leftarrow \rho\mu$.
 - (b) For at most 100 inner iterations:
 - i. Set $X = \text{diag}(z) + \epsilon I$.
 - ii. Solve $(AX^2A^T)\lambda = AX^2c - \mu AXe$.
 - iii. Compute $p = Xe + \frac{1}{\mu}X^2(A^T\lambda - c)$.
 - iv. Update $z \leftarrow z + p$.
 - v. If $\|p\|_2 < 10^{-9}$, exit the inner loop.
3. Threshold small LP-variable entries and reconstruct the signed vector from the two halves of z .

B Selected MATLAB Source

The following listing contains the main solver implementation used in the portfolio version.

```

1 function [z, history] = interior_point_solver(c, A, b, z0, options)
2 %INTERIOR_POINT_SOLVER Solve the LP with a custom Interior Point iteration
3 %
4 %   minimize    c' * z
5 %   subject to  A * z = b
6 %
7 % Inputs:
8 %   c          - objective vector
9 %   A          - equality-constraint matrix
10 %   b          - equality-constraint right-hand side
11 %   z0         - initial estimate
12 %   options    - struct with optional fields:
13 %               mu0, rho, epsilon, max_outer_iterations,
14 %               max_inner_iterations, tolerance, alpha

```

```

15 %
16 % Outputs:
17 %     z           - final LP variable estimate
18 %     history     - iteration diagnostics
19 %
20 % The update follows the algorithm specified for the original project:
21 %
22 %     X = diag(z) + epsilon*I
23 %
24 %     (A*X^2*A')*lambda = A*X^2*c - mu*A*X*e
25 %
26 %     p = X*e + (1/mu)*X^2*(A'*lambda - c)
27 %
28 %     z <- z + alpha*p
29 %
30 % With alpha = 1, the method uses pure Newton iteration.
31
32     if nargin < 5
33         options = struct();
34     end
35
36     options = set_default(options, 'mu0', 10);
37     options = set_default(options, 'rho', 0.6);
38     options = set_default(options, 'epsilon', 0.001);
39     options = set_default(options, 'max_outer_iterations', 20);
40     options = set_default(options, 'max_inner_iterations', 100);
41     options = set_default(options, 'tolerance', 1e-9);
42     options = set_default(options, 'alpha', 1);
43
44     n = numel(z0);
45
46     if size(A, 2) ~= n
47         error('A must have one column per element of z0.');
```

```

69
70     for outer = 1:options.max_outer_iterations
71         if outer > 1
72             mu = mu * options.rho;
73         end
74
75         history.mu(outer) = mu;
76         step_norm = NaN;
77
78         for inner = 1:options.max_inner_iterations
79             X = diag(z) + options.epsilon * I;
80             X2 = X * X;
81
82             lhs = A * X2 * A';
83             rhs = A * X2 * c - mu * A * X * e;
84
85             % MATLAB's backslash operator solves the linear system without
86             % explicitly forming a matrix inverse.
87             lambda = lhs \ rhs;
88
89             p = X * e + (1 / mu) * X2 * (A' * lambda - c);
90             z = z + options.alpha * p;
91
92             step_norm = norm(p, 2);
93             history.total_newton_steps = ...
94                 history.total_newton_steps + 1;
95
96             if step_norm < options.tolerance
97                 break;
98             end
99         end
100
101         history.inner_iterations(outer) = inner;
102         history.final_step_norm(outer) = step_norm;
103     end
104
105     history.outer_iterations = options.max_outer_iterations;
106 end
107
108 function options = set_default(options, field_name, default_value)
109 %SET_DEFAULT Add a default option only when the field is missing or empty.
110     if ~isfield(options, field_name) || isempty(options.(field_name))
111         options.(field_name) = default_value;
112     end
113 end

```

Listing 7: Selected source: interior_point_solver.m